LeetCode 918 — Maximum Sum Circular Subarray

1. Problem Title & Link

Title: LeetCode 918: Maximum Sum Circular Subarray

Link: https://leetcode.com/problems/maximum-sum-circular-subarray/

2. Problem Statement (Short Summary)

Given a circular integer array nums, return the maximum possible sum of a non-empty subarray.

A circular array means:

Last element connects back to first element.

So a subarray may:

1. Stay completely inside the array.
2. Wrap around from the end to the beginning.

3. Examples (Input → Output)

Example 1

Input:

nums = [1,-2,3,-2]

Output:

3

Explanation:

[3]

Maximum sum is 3.

Example 2

Input:

nums = [5,-3,5]

Output:

10

Explanation:

[5] + [5]

Wrap-around subarray:

5 → (skip -3) → 5

Sum:

10

Example 3

Input:

nums = [-3,-2,-3]

Output:

-2

Explanation:

All numbers are negative.

Choose largest element.

4. Constraints

1 <= nums.length <= 30000

-30000 <= nums[i] <= 30000

Important:

Subarray must contain at least one element.

5. Core Concept (Pattern / Topic)

Kadane's Algorithm

Prefix Sum Thinking

Circular Array Optimization

This is one of the most important advanced Kadane problems.

6. Thought Process (Step-by-Step Explanation)

Step 1: Normal Maximum Subarray

Ignoring circular behavior:

nums = [1,-2,3,-2]

Maximum subarray:

[3]

Answer:

3

This is standard Kadane.

Step 2: Circular Possibility

Consider:

[5,-3,5]

Normal Kadane:

5 + (-3) + 5 = 7

But circularly:

5 → wrap → 5

Sum:

10

Better answer.

Key Insight

Circular Maximum Subarray means:

Take entire array

Remove minimum subarray

Example:

[5,-3,5]

Total Sum:

7

Minimum Subarray:

[-3]

Circular Answer:

7 - (-3)

= 10

Thus answer becomes:

max(

Normal Maximum Subarray,

Total Sum - Minimum Subarray

)

7. Visual / Intuition Diagram (ASCII)

```
Example:
[5,-3,5]
Circular View:
      5
    /   \
  -3     5
Take:
5 + 5
Skip:
-3
Equivalent to:
Total Sum
-
Minimum Subarray
```

8. Pseudocode

```
Find totalSum

Find maximum subarray using Kadane

Find minimum subarray using Kadane

If maxSubarray < 0
    return maxSubarray

return max(
        maxSubarray,
        totalSum - minSubarray
)
```

9. Code Implementation

Python

```python
class Solution:
    def maxSubarraySumCircular(self, nums):

        total = sum(nums)

        curMax = 0
        maxSum = nums[0]

        curMin = 0
        minSum = nums[0]

        for num in nums:

            curMax = max(curMax + num, num)
            maxSum = max(maxSum, curMax)

            curMin = min(curMin + num, num)
            minSum = min(minSum, curMin)

        if maxSum < 0:
            return maxSum

        return max(maxSum, total - minSum)
```

Java

```java
class Solution {

    public int maxSubarraySumCircular(int[] nums) {
```

```
        int total = 0;

        int curMax = 0;
        int maxSum = nums[0];

        int curMin = 0;
        int minSum = nums[0];

        for(int num : nums) {

            total += num;

            curMax = Math.max(curMax + num, num);
            maxSum = Math.max(maxSum, curMax);

            curMin = Math.min(curMin + num, num);
            minSum = Math.min(minSum, curMin);
        }

        if(maxSum < 0) {
            return maxSum;
        }

        return Math.max(maxSum, total - minSum);
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

Single traversal.

Space Complexity

O(1)

Only variables used.

11. Common Mistakes / Edge Cases

Mistake 1

Using only Kadane.

Fails for:

[5,-3,5]

Expected:

10

Kadane gives:

7

Mistake 2

Ignoring circular behavior.

Mistake 3

Not handling all-negative arrays.

Example:

[-3,-2,-3]

Wrong:

0

Correct:

-2

12. Detailed Dry Run

Input:

nums = [5,-3,5]

Total Sum

7

Maximum Subarray

Num	Current Max	Max Sum
5	5	5
-3	2	5
5	7	7

Result:

maxSum = 7

Minimum Subarray

Num	Current Min	Min Sum
5	5	5
-3	-3	-3
5	2	-3

Result:

minSum = -3

Circular Answer

total - minSum

7 - (-3)

= 10

Final Answer:

max(7,10)

= 10

13. Common Use Cases (Real-Life / Interview)

Circular Scheduling

Examples:

24-hour monitoring systems

CPU scheduling

Circular buffers

Network ring topology

Round-robin processing

Sometimes best segment wraps around the end.

14. Common Traps (Important!)

Trap 1

Thinking circular means duplicate array.

Not necessary.

Trap 2

Forgetting:

Circular Maximum

=

Total Sum - Minimum Subarray

Trap 3

Missing all-negative case.

Most common interview mistake.

15. Builds To (Related LeetCode Problems)

LC 53 Maximum Subarray

↓

LC 1413 Minimum Start Value

↓

LC 918 Maximum Circular Subarray

↓

LC 152 Maximum Product Subarray

↓

LC 1749 Maximum Absolute Sum

Kadane Roadmap.

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^2)$	O(1)	Too slow
Prefix Sum	$O(n^2)$	O(n)	Better
Kadane + Min Kadane	O(n)	O(1)	Optimal

Approach 1: Brute Force

Generate every circular subarray.

$O(n^2)$

Not practical.

Approach 2: Prefix Sum

Use doubled array.

Works but more complex.

Approach 3: Kadane + Minimum Kadane

Most elegant and optimal.

17. Why This Solution Works (Short Intuition)

A circular maximum subarray has two possibilities:

Case 1

Normal subarray.

Handled by Kadane.

Case 2

Wrapped subarray.

Equivalent to:

Entire Array

-

Minimum Subarray

Taking the best of these two cases guarantees the maximum possible answer.

18. Variations / Follow-Up Questions

Follow-Up 1

Find minimum circular subarray.

Follow-Up 2

Return indices of the circular subarray.

Follow-Up 3

Maximum product circular subarray.

Much harder.

Follow-Up 4

What if array updates continuously?

Leads to segment trees.

Pattern Learned

Kadane's Algorithm

Maximum Subarray

+

Minimum Subarray

+

Total Sum

↓

Maximum Circular Subarray

Answer

```
max(
   maxSubarray,
   totalSum - minSubarray
```